



Estácio

Universidade Estácio de Sá

-
- **DESENVOLVIMENTO FULL STACK**
 - **Disciplina: RPG0015 - Vamos Manter as Informações?**

Repositorio Git:

<https://github.com/matheusg999/modImplementacaoSQL.git>

- **Matheus Gomes Silva - MATRICULA: 202401478822**
-

Missão Prática | Nível 2 | Mundo 3

Modelagem e implementação de um banco de dados simples, utilizando como base o SQL Server.

Procedimento 1: Criando o Banco de Dados Procedimento 2: Alimentando a Base

 **Objetivos da Prática**

- **Identificar os requisitos de um sistema e transformá-los no modelo adequado.**
- **Utilizar ferramentas de modelagem para bases de dados relacionais.**
- **Explorar a sintaxe SQL na criação das estruturas do banco (DDL).**
- **Explorar a sintaxe SQL na consulta e manipulação de dados (DML).**
- **No final do exercício, o aluno terá vivenciado a experiência de modelar a base de dados para um sistema simples, além de implementá-la, através da sintaxe SQL, na plataforma do SQL Server**

Códigos

Procedimento 1: Criando o Banco de Dados

```
CREATE TABLE produto (  
    idproduto INTEGER NOT NULL ,  
    nome VARCHAR(255) ,  
    quantidade INTEGER ,  
    precoVenda NUMERIC(5,2) ,  
    PRIMARY KEY(idproduto));
```

```
CREATE TABLE usuario (  
    idusuario INTEGER NOT NULL ,  
    nomeUsuario VARCHAR(255) ,  
    senha VARCHAR(255) ,  
    PRIMARY KEY(idusuario));
```

```
CREATE TABLE pessoa (  
    idpessoa INTEGER NOT NULL ,  
    nome VARCHAR(255) ,  
    logradouro VARCHAR(255) ,  
    cidade VARCHAR(255) ,  
    estado VARCHAR(2) ,  
    telefone VARCHAR(15) ,  
    email VARCHAR(255) ,  
    cpf_cnpj VARCHAR(18) ,  
    tipoPessoa CHAR(1) ,
```

```
PRIMARY KEY(idpessoa));  
  
CREATE TABLE movimento (  
    idmovimento INTEGER NOT NULL ,  
    usuario_idusuario INTEGER NOT NULL ,  
    pessoa_idpessoa INTEGER ,  
    produto_idproduto INTEGER ,  
    tipoMovimento CHAR(1) ,  
    quantidade INTEGER ,  
    valorUnitario NUMERIC(5,2) ,  
    PRIMARY KEY(idmovimento) ,  
    FOREIGN KEY(produto_idproduto)  
        REFERENCES produto(idproduto),  
    FOREIGN KEY(pessoa_idpessoa)  
        REFERENCES pessoa(idpessoa),  
    FOREIGN KEY(usuario_idusuario)  
        REFERENCES usuario(idusuario));
```

```
create sequence seq_Pessoa  
as numeric  
start with 1  
increment by 1  
no cycle;
```

Procedimento 2: Alimentando a Base

insert into usuario values (1, 'op1', 'op1'),(2, 'op2', 'op2');

insert into produto values (1, 'Banana', 100, 5.00),(3, 'Laranja', 500, 2.00),(4, 'Manga', 800, 4.00);

INSERT INTO pessoa VALUES (NEXT VALUE FOR seq_pessoa_id, 'Joao', 'Rua 12, cas 3, Quitanda', 'Riacho do Sul', 'PA', '1111-1111', 'joao@riacho.com', '1111111111', 'F');

INSERT INTO pessoa VALUES (NEXT VALUE FOR seq_pessoa_id, 'JJC', 'Rua 11, Centro', 'Riacho do Norte', 'PA', '1212-1212', 'jjc@riacho.com', '22222222222222', 'J');

INSERT INTO movimento VALUES (1,1,1,1,'S',20,4.00), (4,1,1,3,'S',15,2.00), (5,2,1,3,'S',10,3.00), (7,1,2,3,'E',15,5.00), (8,1,2,4,'E',20,4.00);

--Dados completos de pessoas físicas. select * from pessoa

where tipoPessoa = 'F';

--Dados completos de pessoas jurídicas. select * from pessoa

where tipoPessoa = 'J';

--Movimentações de entrada, com produto, fornecedor, quantidade, preço unitário e valor total. SELECT idmovimento, produto_idproduto, produto.nome AS 'Produto', pessoa_idpessoa, pessoa.nome AS 'Fornecedor', movimento.quantidade, valorUnitario, (movimento.quantidade * valorUnitario) AS valor_total FROM movimento JOIN pessoa ON movimento.pessoa_idpessoa = pessoa.idpessoa JOIN produto ON movimento.produto_idproduto = produto.idproduto WHERE movimento.tipoMovimento = 'E';

-- Movimentações de saída, com produto, comprador, quantidade, preço unitário e valor total SELECT idmovimento, produto_idproduto, produto.nome AS Produto, pessoa_idpessoa, pessoa.nome AS Comprador, movimento.quantidade, valorUnitario, (movimento.quantidade * valorUnitario) AS valor_total FROM movimento JOIN pessoa ON movimento.pessoa_idpessoa = pessoa.idpessoa JOIN produto ON movimento.produto_idproduto = produto.idproduto WHERE movimento.tipoMovimento = 'S';

--Valor total das entradas agrupadas por produto. SELECT produto.nome, SUM(movimento.quantidade * movimento.valorUnitario) AS ValorTotalEntradas FROM movimento JOIN produto ON produto.idproduto = movimento.produto_idproduto WHERE movimento.tipoMovimento = 'E' GROUP BY produto.nome;

--Valor total das saídas agrupadas por produto. SELECT produto.nome, SUM(movimento.quantidade * movimento.valorUnitario) AS ValorTotalSaidas FROM

**movimento JOIN produto ON produto.idproduto = movimento.produto_idproduto
WHERE movimento.tipoMovimento = 'S' GROUP BY produto.nome;**

**-- Operadores que não efetuaram movimentações de entrada (compra). SELECT
movimento.usuario_idusuario AS ID_DO_OPERADOR FROM movimento EXCEPT
SELECT movimento.usuario_idusuario FROM movimento WHERE
movimento.tipoMovimento = 'E';**

**-- Valor total de entrada, agrupado por operador. SELECT usuario.nomeUsuario AS
OPERADOR, SUM(movimento.quantidade * movimento.valorUnitario) AS
ValorTotalEntradas FROM movimento JOIN usuario ON usuario.idusuario =
movimento.usuario_idusuario WHERE movimento.tipoMovimento = 'E' GROUP BY
usuario.nomeUsuario;**

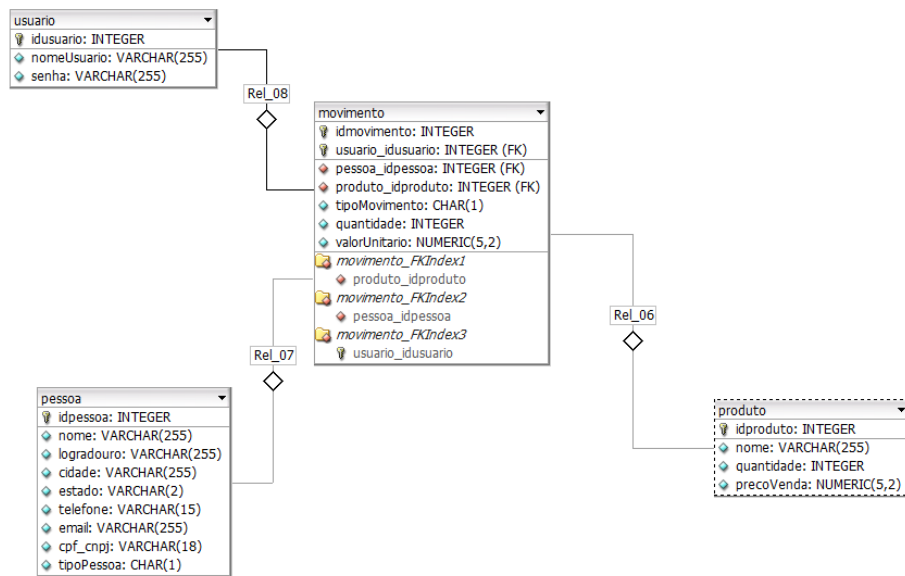
**-- Valor total de saída, agrupado por operador. SELECT usuario.nomeUsuario AS
OPERADOR, SUM(movimento.quantidade * movimento.valorUnitario) AS
ValorTotalSaidas FROM movimento JOIN usuario ON usuario.idusuario =
movimento.usuario_idusuario WHERE movimento.tipoMovimento = 'S' GROUP BY
usuario.nomeUsuario;**

**-- Valor médio de venda por produto, utilizando média ponderada. SELECT
produto.nome, SUM(movimento.quantidade * movimento.valorUnitario) /
SUM(movimento.quantidade) AS ValorMedioDeVenda FROM movimento JOIN
produto ON produto.idproduto = movimento.produto_idproduto WHERE
movimento.tipoMovimento = 'S' GROUP BY produto.nome;**

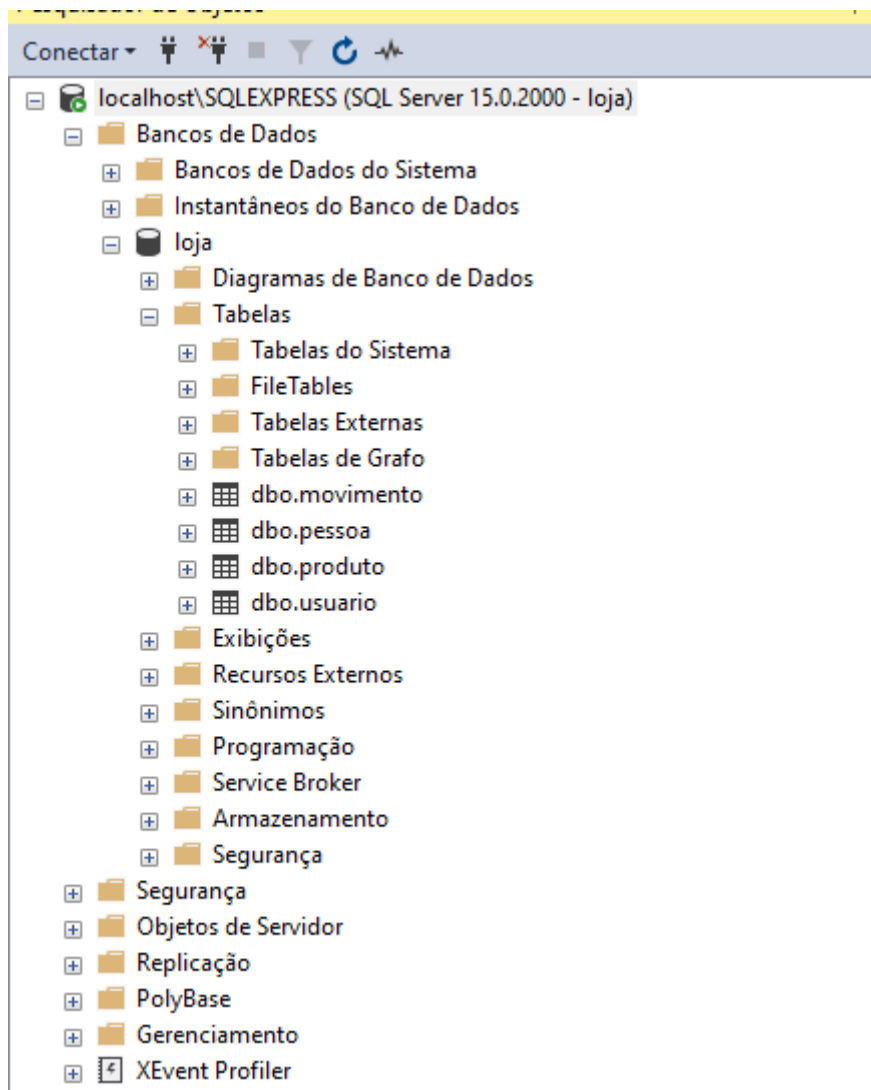
Resultados:

 **Procedimento 1:**

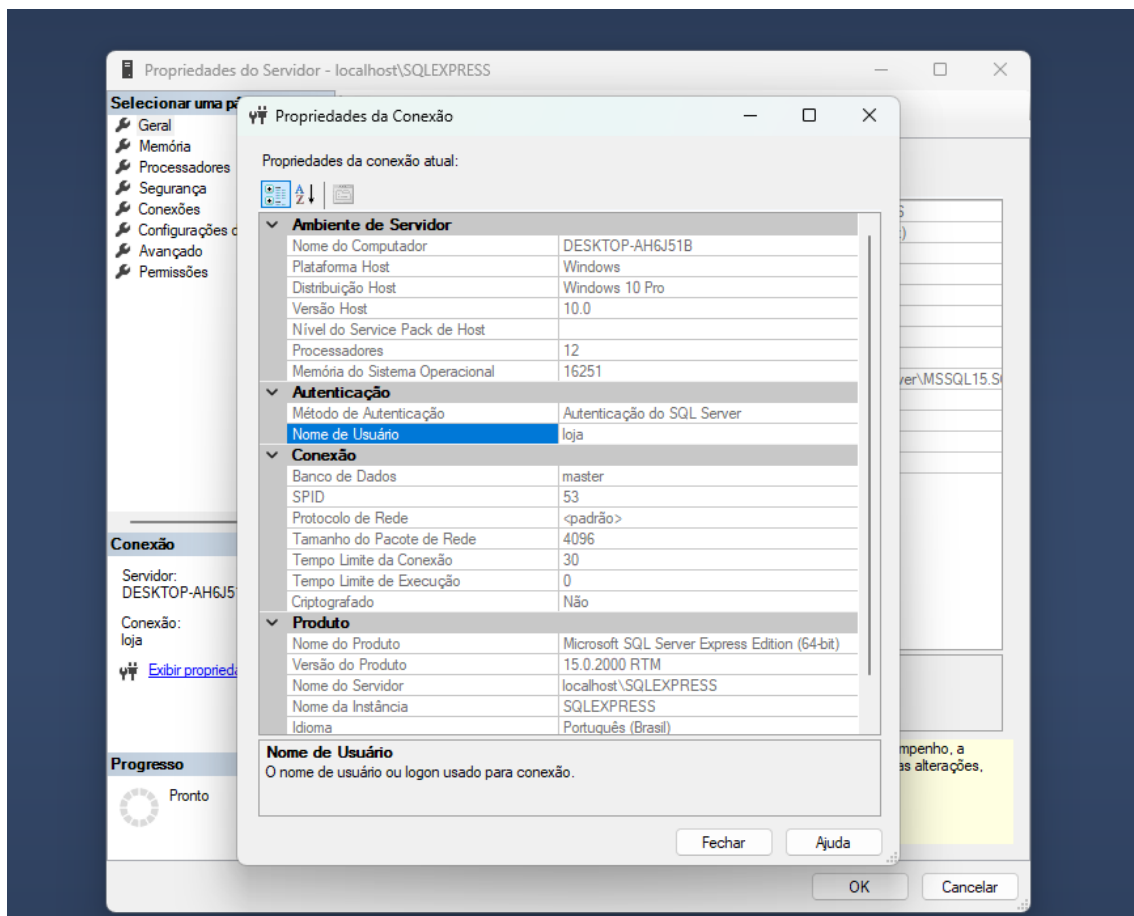
Modelo



Banco e Tabelas



Usuario/Logon Loja



Conectar ao Servidor

SQL Server

LogonPropriedades da ConexãoAlways EncryptedParâmetros de Conexão Adicionais

Servidor

Tipo de servidor:

Mecanismo de Banco de Dados

Nome do servidor:

localhost\SQLEXPRESS

Autenticação:

Autenticação do SQL Server

Logon:

loja

Senha:

☒ Lembrar senha

Segurança de Conexão

Criptografia:

Opcional

☐ Certificado do Servidor Confiável

Nome do host no certificado:

Conectar

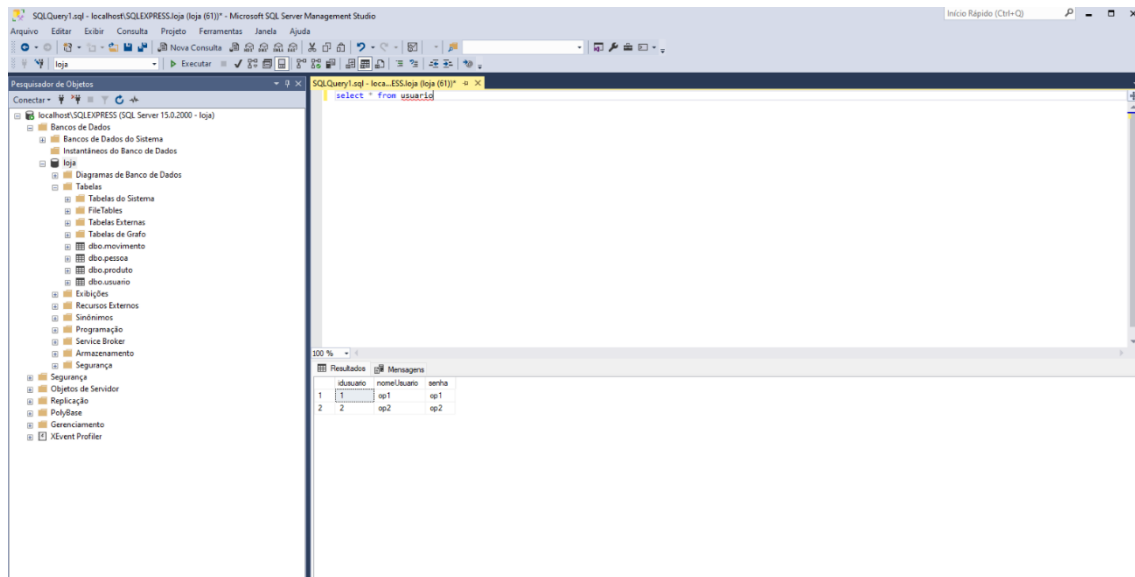
Cancelar

Ajuda

Opções <<

▶ Procedimento 2:

Usuarios



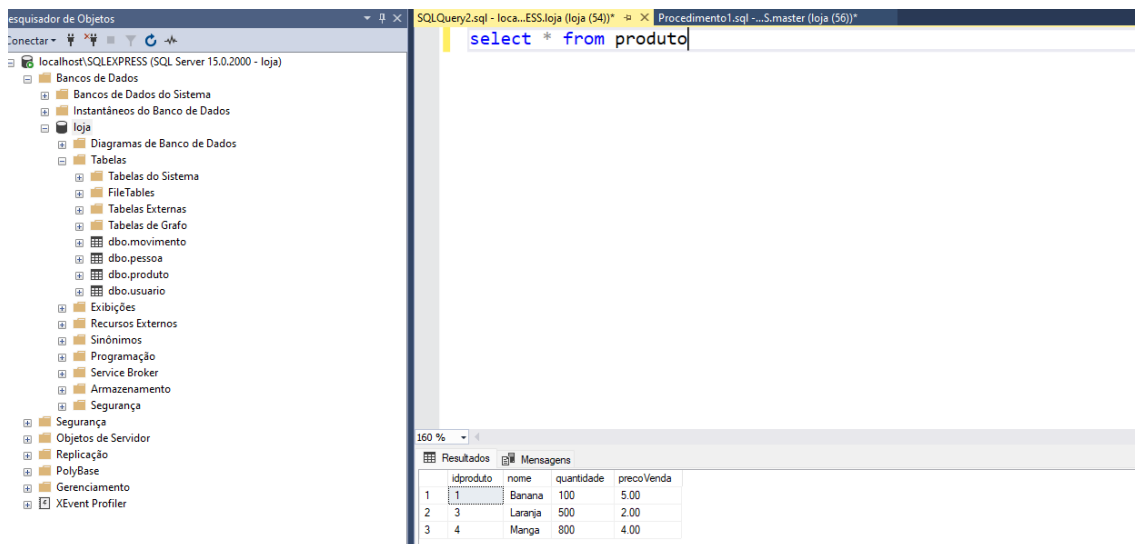
SQLQuery1.sql - localhost\SQLEXPRESS\loja (81)* - Microsoft SQL Server Management Studio

SQLQuery1.sql - loja...ESS\loja (81)*

```
select * from usuario
```

	idusuario	nomeusuario	senha	op1	op2
1	1	op1	op1		
2	2	op2	op2		

Produtos



esquisador de Objetos

SQLQuery2.sql - loja...ESS\loja (54)*

```
select * from produto
```

	idproduto	nome	quantidade	precoVenda
1	1	Banana	100	5.00
2	3	Laranja	500	2.00
3	4	Manga	800	4.00

Movimentos

SQLQuery1.sql - localhost\SQLEXPRESS:loja (loja (54)) - Microsoft SQL Server Management Studio

Arquivo Editar Exibir Consulta Projeto Ferramentas Janela Ajuda

loja

SQLQuery1.sql - loja_ESS:loja (loja (54))

```
select * from movimento
```

100 %

	idmovimento	usuario_idusuario	pessoa_idpessoa	produto_idproduto	tipoMovimento	quantidade	valorInicial
1	1	1	1	1	S	20	4.00
2	4	1	1	3	S	15	2.00
3	5	2	1	3	S	10	3.00
4	7	1	2	3	E	15	5.00
5	8	1	2	4	E	20	4.00

Dados completos de pessoas físicas e jurídicas

```
--Dados completos de pessoas físicas.
select * from pessoa
where tipoPessoa = 'F';
```

100 %

	idpessoa	nome	logradouro	cidade	estado	telefone	email	cpf_cnpj	tipoPessoa
1	1	Joao	Rua 12, cas 3, Quitanda	Riacho do Sul	PA	1111-1111	joao@riacho.com	1111111111111	F

```
--Dados completos de pessoas jurídicas.
select * from pessoa
where tipoPessoa = 'J';
```

100 %

	idpessoa	nome	logradouro	cidade	estado	telefone	email	cpf_cnpj	tipoPessoa
1	2	JJC	Rua 11, Centro	Riacho do Norte	PA	1212-1212	jjc@riacho.com	222222222222222	J

Movimentações de entrada e saída, com produto, comprador, fornecedor, quantidade, preço unitário e valor total

--Movimentações de entrada, com produto, fornecedor, quantidade, preço unitário e valor total.

SELECT

idmovimento,

produto_idproduto,

produto.nome AS 'Produto',

pessoa_idpessoa,

pessoa.nome AS 'Fornecedor',

movimento.quantidade,

valorUnitario,

(movimento.quantidade * valorUnitario) AS valor_total

FROM movimento

JOIN pessoa

ON movimento.pessoa_idpessoa = pessoa.idpessoa

JOIN produto

ON movimento.produto_idproduto = produto.idproduto

WHERE movimento.tipoMovimento = 'E';

100 %

Resultados Mensagens

	idmovimento	produto_idproduto	Produto	pessoa_idpessoa	Fornecedor	quantidade	valorUnitario	valor_total
1	7	3	Laranja	2	JJC	15	5.00	75.00
2	8	4	Manga	2	JJC	20	4.00	80.00

-- Movimentações de saída, com produto, comprador, quantidade, preço unitário e valor total

SELECT

idmovimento,

produto_idproduto,

produto.nome AS Produto,

pessoa_idpessoa,

pessoa.nome AS Comprador,

movimento.quantidade,

valorUnitario,

(movimento.quantidade * valorUnitario) AS valor_total

FROM movimento

JOIN pessoa

ON movimento.pessoa_idpessoa = pessoa.idpessoa

JOIN produto

ON movimento.produto_idproduto = produto.idproduto

WHERE movimento.tipoMovimento = 'S';

100 %

Resultados Mensagens

	idmovimento	produto_idproduto	Produto	pessoa_idpessoa	Comprador	quantidade	valorUnitario	valor_total
1	1	1	Banana	1	Joao	20	4.00	80.00
2	4	3	Laranja	1	Joao	15	2.00	30.00
3	5	3	Laranja	1	Joao	10	3.00	30.00

Valor total das entradas e saídas agrupadas por produto

```
--Valor total das entradas agrupadas por produto.
SELECT
    produto.nome,
    SUM(movimento.quantidade * movimento.valorUnitario) AS ValorTotalEntradas
FROM movimento
JOIN produto
    ON produto.idproduto = movimento.produto_idproduto
WHERE movimento.tipoMovimento = 'E'
GROUP BY produto.nome;

--Valor total das saídas agrupadas por produto.
SELECT
    produto.nome,
    SUM(movimento.quantidade * movimento.valorUnitario) AS ValorTotalSaídas
FROM movimento
JOIN produto
    ON produto.idproduto = movimento.produto_idproduto
WHERE movimento.tipoMovimento = 'S'
GROUP BY produto.nome;
```

100 %

Resultados		Mensagens	
	nome	ValorTotalEntradas	
1	Laranja	75.00	
2	Manga	80.00	

	nome	ValorTotalSaídas	
1	Banana	80.00	
2	Laranja	60.00	

Operadores que não efetuaram movimentações de entrada (compra). Valor total de entrada e saída agrupado por operador

000 - loja)

```
Procedimento1.sql - ...ESS.Loja (loja (54)) -
```

```
JOIN produto
    ON produto.idproduto = movimento.produto_idproduto
WHERE movimento.tipoMovimento = 'S'
GROUP BY produto.nome;

-- Operadores que não efetuaram movimentações de entrada (compra).
SELECT movimento.usuario_idusuario AS ID_DO_OPERADOR
FROM movimento
EXCEPT
SELECT movimento.usuario_idusuario
FROM movimento
WHERE movimento.tipoMovimento = 'E';

-- Valor total de entrada, agrupado por operador.
SELECT
    usuario.nomeusuario AS OPERADOR,
    SUM(movimento.quantidade * movimento.valorUnitario) AS ValorTotalEntradas
FROM movimento
JOIN usuario
    ON usuario.idusuario = movimento.usuario_idusuario
WHERE movimento.tipoMovimento = 'E'
GROUP BY usuario.nomeusuario;

-- Valor total de saída, agrupado por operador.
SELECT
    usuario.nomeusuario AS OPERADOR,
    SUM(movimento.quantidade * movimento.valorUnitario) AS ValorTotalSaídas
FROM movimento
JOIN usuario
    ON usuario.idusuario = movimento.usuario_idusuario
WHERE movimento.tipoMovimento = 'S'
GROUP BY usuario.nomeusuario;
```

83 %

Resultados		Mensagens	
	ID_DO_OPERADOR		
1	2		

	OPERADOR	ValorTotalEntradas	
1	op1	155.00	

	OPERADOR	ValorTotalSaídas	
1	op1	110.00	
2	op2	30.00	

Valor médio de venda por produto, utilizando média ponderada

```
-- Valor médio de venda por produto, utilizando média ponderada.
SELECT
    produto.nome,
    SUM(movimento.quantidade * movimento.valorUnitario) / SUM(movimento.quantidade) AS ValorMedioDeVenda
FROM movimento
JOIN produto
    ON produto.idproduto = movimento.produto_idproduto
WHERE movimento.tipoMovimento = 'S'
GROUP BY produto.nome;
```

83 %

Resultados Mensagens

	nome	ValorMedioDeVenda
1	Banana	4.000000
2	Laranja	2.400000

Análise e Conclusão

Como são implementadas as diferentes cardinalidades, basicamente 1x1, 1xN ou NxN, em um banco de dados relacional?

Através dos graus de relação que entidades ou tabelas têm entre si.

Que tipo de relacionamento deve ser utilizado para representar o uso de herança em bancos de dados relacionais?

Relacionamento 1x1.

Como o SQL Server Management Studio permite a melhoria da produtividade nas tarefas relacionadas ao gerenciamento do banco de dados?

Através de um editor de consultas, monitoramento de desempenho, segurança e gerenciamento de permissões.

Quais as diferenças no uso de SEQUENCE e IDENTITY?

As SEQUENCES são acionadas sempre que necessário, sem dependência de tabelas e campos no banco, podendo ser chamadas diretamente por aplicativos. Com SEQUENCES, é possível obter o novo valor antes de usá-lo em um comando, diferente do IDENTITY, que não permite isso.

Além disso, com IDENTITY não é possível gerar novos valores em uma instrução UPDATE, enquanto com SEQUENCE isso é viável. As SEQUENCES permitem definir valores máximos e mínimos, trabalhar de forma cíclica e com cache, e obter vários valores em sequência de uma só vez, utilizando a procedure SP_SEQUENCE_GET_RANGE, o que melhora o desempenho.

Uma das grandes vantagens do IDENTITY é sua integração com transações de INSERT, pois o próximo valor só é gerado quando o comando é executado com sucesso. Já com SEQUENCE, o valor é alterado mesmo que a transação falhe.

Qual a importância das chaves estrangeiras para a consistência do banco?

A utilização da chave estrangeira possibilita a implementação da integridade de dados diretamente no banco de dados, conhecida como integridade referencial. Uma chave estrangeira representa o relacionamento entre tabelas.

Quais operadores do SQL pertencem à álgebra relacional e quais são definidos no cálculo relacional?

Operadores da álgebra relacional:

- Seleção
- Restrição
- Projeção
- União
- Interseção
- Diferença de conjuntos
- Produto cartesiano
- Junção
- Divisão
- Renomeação
- Atribuição

Operadores do cálculo relacional:

- Igual
- Diferente
- Maior
- Menor
- Maior ou igual
- Menor ou igual

Como é feito o agrupamento em consultas, e qual requisito é obrigatório?

O agrupamento em consultas é feito utilizando o comando GROUP BY.